



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий
Кафедра информационных технологий в атомной энергетике

ОТЧЕТ ПО ПРАКТИКЕ №6
«Разработка автоматизированной системы управления
материально-техническим обеспечением в сфере лыжного
спорта»

по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Принял преподаватель

Щербаков К. В.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

1. Анализ функций АСУ и определение функции для доработки

На основе анализа функциональных требований (ПР2) и диаграмм последовательности (ПР4) был выделен ряд функций, реализация которых в типовых ERP-системах требует адаптации под специфику лыжного спорта.

Выбранная функция для разработки: Функция 2.6 — "Выдача инвентаря спортсмену" (из Модуля 2: Складской учет, логистика и управление имуществом)

Обоснование выбора:

1. Специфичность предметной области — лыжный инвентарь требует учета размерных характеристик (ростовка, жесткость, размер креплений), что усложняет стандартную выдачу
2. Бизнес-логика с несколькими условиями — функция включает:
 - Проверку спортсмена на наличие задолженностей
 - Подбор инвентаря по размерным характеристикам
 - Учет сезонности (разные типы инвентаря для разных стилей катания)
 - Фиксацию даты планируемого возврата
3. Наличие альтернативных потоков — согласно диаграмме последовательности из ПР4, функция имеет альтернативный сценарий (блокировка выдачи должникам)
4. Высокая частота использования — в сезон активных тренировок функция будет использоваться ежедневно
5. Требование к интеграции с оборудованием — работа со сканером штрих-кода требует специализированной логики

2. Анализ существующих языков программирования и IDE

Таблица 1 – Рассмотренные языки программирования

Язык	Преимущества	Недостатки	Применимость к задаче
Java	Кроссплатформенность, обширная экосистема, Spring Framework, строгая типизация	Более объемный код, выше порог входа	Высокая — подходит для серверной части АСУ
Python	Быстрая разработка, простота синтаксиса, богатые библиотеки для работы с данными	Динамическая типизация, ниже производительность	Средняя — может использоваться для прототипирования
C# (.NET)	Высокая производительность, интеграция с Windows, современный синтаксис	Привязка к экосистеме Microsoft (хотя .NET Core кроссплатформенный)	Средняя — требует дополнительной настройки под Linux
Kotlin	Современный синтаксис, полная совместимость с Java, безопасная работа с null	Меньше специалистов на рынке	Высокая — хорошая альтернатива Java
TypeScript	Строгая типизация, отличная поддержка в браузерах, единый стек с фронтендом	Не подходит для сложной бизнес-логики на сервере	Низкая — лучше использовать для клиентской части

Таблица 2 – Рассмотренные IDE

IDE	Поддерживаемые языки	Преимущества	Недостатки
IntelliJ IDEA	Java, Kotlin, Python, SQL, JavaScript	Мощный рефакторинг, отличная поддержка Spring, встроенные инструменты БД	Платная (Ultimate), ресурсоемкая
Visual Studio Code	TypeScript, Python, Java (через плагины), JavaScript	Бесплатный, легковесный, огромное расширение плагинов	Требует настройки для Java, меньше встроенных инструментов
Eclipse	Java, C/C++, PHP	Бесплатный, открытый код, проверенный временем	Устаревший интерфейс, медленнее IntelliJ
PyCharm	Python, JavaScript, SQL	Лучшая IDE для Python, встроенные инструменты для научных вычислений	Платная (Professional), специализирована под Python
Visual Studio	C#, .NET, C++	Мощный отладчик, отличная интеграция с .NET	Тяжеловесная, привязка к Windows

3 Обоснованный выбор языка, IDE и библиотек

Выбранный язык программирования: Java 21 (LTS)

Обоснование:

1. Соответствие архитектуре — в ПП5 выбрана платформа Spring Boot 3.4 на Java 21
2. Кроссплатформенность — возможность развертывания на российских ОС (Astra Linux)
3. Строгая типизация — минимизация ошибок в критической логике выдачи инвентаря
4. Поддержка виртуальных потоков (Project Loom) — высокая производительность при параллельной выдаче инвентаря
5. Обширная экосистема — готовые решения для работы со сканерами штрих-кодов, БД, REST API

Выбранная IDE: IntelliJ IDEA 2024.3 (Ultimate Edition)

Обоснование:

1. Лучшая поддержка Spring Boot — автодополнение, генерация кода, встроенный HTTP-клиент
2. Интеграция с JPA/Hibernate — визуальное редактирование сущностей и запросов
3. Встроенные инструменты для работы с БД — просмотр таблиц, выполнение SQL-запросов без переключения
4. Поддержка Java 21 — все новые возможности языка
5. Git-интеграция — удобная работа с системой контроля версий

Таблица 3 – Выбранные библиотеки

Библиотека	Версия	Назначение	Обоснование
Spring Boot Starter Web	3.4.0	Создание REST API	Стандарт для веб-приложений на Java
Spring Boot Starter Data JPA	3.4.0	Работа с БД (ORM)	Упрощение операций с PostgreSQL
Spring Boot Starter Validation	3.4.0	Валидация входных данных	Проверка корректности данных выдачи

PostgreSQL JDBC Driver	42.7.0	Подключение к PostgreSQL	Официальный драйвер для Postgres Pro
Lombok	1.18.34	Генерация кода (геттеры, сеттеры)	Сокращение boilerplate-кода
MapStruct	1.6.0	Маппинг между DTO и Entity	Трансформация данных между слоями
Spring Boot Starter Test	3.4.0	Модульное тестирование	Проверка корректности бизнес-логики
ZXing (Zebra Crossing)	3.5.3	Генерация и сканирование штрих-кодов	Работа со штрих-кодами инвентаря
Jackson	2.17.0	Сериализация/десериализация JSON	Встроена в Spring Boot, обработка API-запросов

4 Генерация программного кода

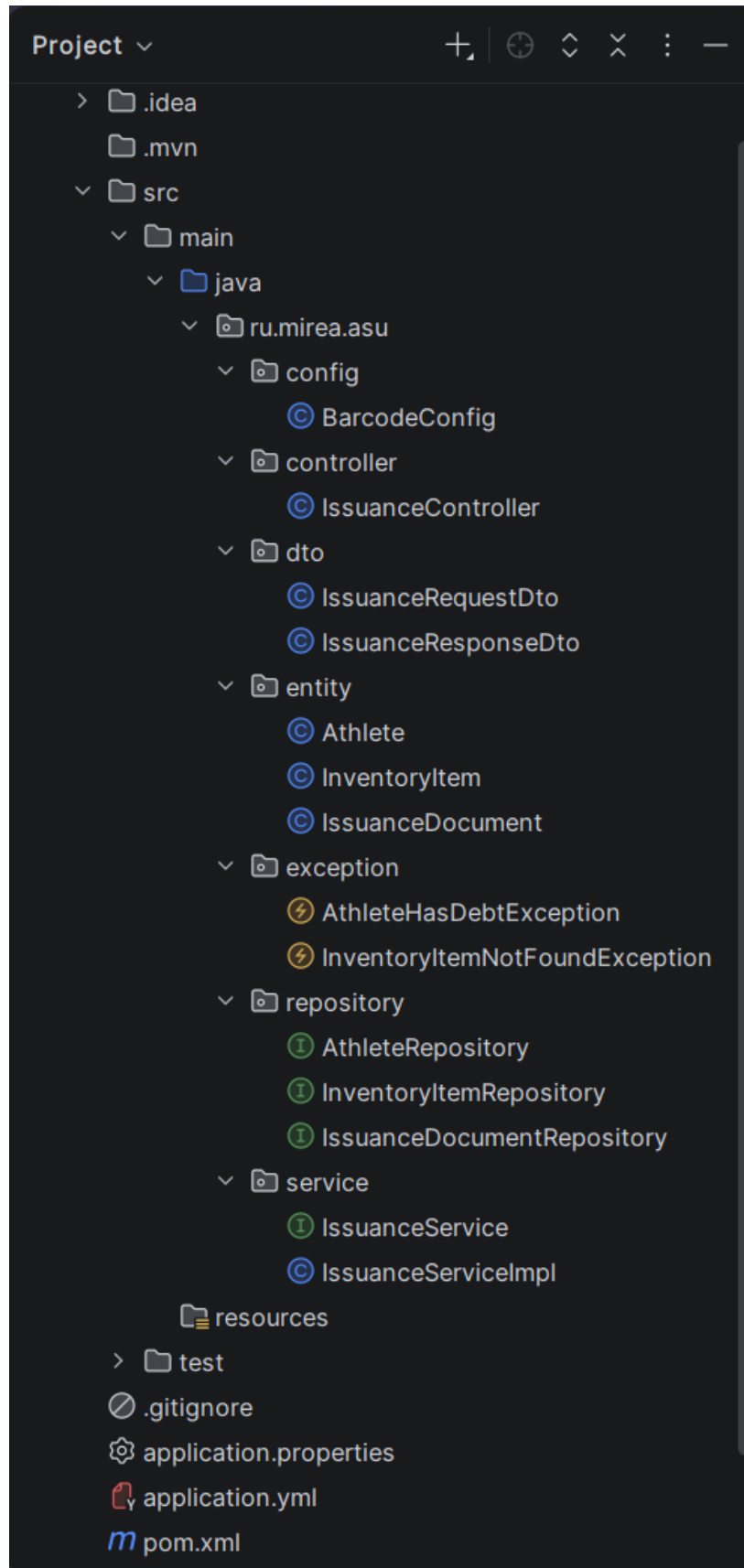


Рисунок 1 – Структура разрабатываемого модуля

Листинг 1 – Файл pom.xml (зависимости Maven)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.0</version>
  </parent>

  <groupId>ru.mirea.asu</groupId>
  <artifactId>inventory-management</artifactId>
  <version>1.0.0</version>
  <name>ASU Inventory Management</name>
  <description>Модуль выдачи инвентаря для АСУ лыжного спорта</description>

  <properties>
    <java.version>21</java.version>
    <lombok.version>1.18.34</lombok.version>
    <mapstruct.version>1.6.0</mapstruct.version>
    <zxing.version>3.5.3</zxing.version>
  </properties>

  <dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- PostgreSQL Driver -->
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.0</version>
    </dependency>

    <!-- Lombok -->
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>${lombok.version}</version>
      <scope>provided</scope>
    </dependency>

    <!-- MapStruct -->
    <dependency>
      <groupId>org.mapstruct</groupId>
      <artifactId>mapstruct</artifactId>
      <version>${mapstruct.version}</version>
    </dependency>
```

```

<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct-processor</artifactId>
  <version>${mapstruct.version}</version>
  <scope>provided</scope>
</dependency>

<!-- ZXing for barcode generation -->
<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>core</artifactId>
  <version>${zxing.version}</version>
</dependency>
<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>javase</artifactId>
  <version>${zxing.version}</version>
</dependency>

<!-- Test dependencies -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <source>21</source>
        <target>21</target>
        <annotationProcessorPaths>
          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>${lombok.version}</version>
          </path>
          <path>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct-processor</artifactId>
            <version>${mapstruct.version}</version>
          </path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

Листинг 2 – Сущность *Athlete.java* (Спортсмен)

```

package ru.mirea.asu.entity;

import jakarta.persistence.*;
import lombok.Data;

```



```

import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

@Entity
@Table(name = "athletes")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Athlete {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id", updatable = false, nullable = false)
    private UUID id;

    @Column(name = "full_name", nullable = false, length = 200)
    private String fullName;

    @Column(name = "birth_date", nullable = false)
    private LocalDate birthDate;

    @Column(name = "sports_category", length = 50)
    private String sportsCategory;

    @Column(name = "coach_id")
    private UUID coachId;

    @Column(name = "phone_number", length = 20)
    private String phoneNumber;

    @Column(name = "email", length = 100)
    private String email;

    @Column(name = "has_active_debt", nullable = false)
    private boolean hasActiveDebt = false;

    @OneToMany(mappedBy = "athlete", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<IssuanceDocument> issuanceDocuments = new ArrayList<>();

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
}

```

Листинг 3 – Сущность InventoryItem.java (Единица инвентаря)

```

package ru.mirea.asu.entity;

import jakarta.persistence.*;
import lombok.Data;

```

```

import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "inventory_items")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class InventoryItem {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id", updatable = false, nullable = false)
    private UUID id;

    @Column(name = "barcode", unique = true, nullable = false, length = 50)
    private String barcode;

    @Column(name = "product_name", nullable = false, length = 200)
    private String productName;

    @Column(name = "size", length = 20)
    private String size;

    @Column(name = "flex_rating", length = 20)
    private String flexRating;

    @Column(name = "binding_type", length = 50)
    private String bindingType;

    @Column(name = "manufacture_date")
    private LocalDate manufactureDate;

    @Column(name = "commissioning_date")
    private LocalDate commissioningDate;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private InventoryStatus status = InventoryStatus.AVAILABLE;

    @Column(name = "current_holder_id")
    private UUID currentHolderId;

    @Column(name = "warehouse_location", length = 100)
    private String warehouseLocation;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    public enum InventoryStatus {
        AVAILABLE,          // доступен на складе
        RESERVED,           // зарезервирован
        ISSUED,              // выдан спортсмену
        UNDER_REPAIR,       // в ремонте
        DEFECTIVE,           // бракован
        WRITTEN_OFF          // списан
    }
}

```

Листинг 4 – Сущность IssuanceDocument.java (Документ выдачи)

```
package ru.mirea.asu.entity;

import jakarta.persistence.*;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "issuance_documents")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class IssuanceDocument {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id", updatable = false, nullable = false)
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "athlete_id", nullable = false)
    private Athlete athlete;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "inventory_item_id", nullable = false)
    private InventoryItem inventoryItem;

    @Column(name = "issuance_date", nullable = false)
    private LocalDate issuanceDate;

    @Column(name = "planned_return_date")
    private LocalDate plannedReturnDate;

    @Column(name = "actual_return_date")
    private LocalDate actualReturnDate;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private IssuanceStatus status = IssuanceStatus.ACTIVE;

    @Column(name = "storekeeper_id", nullable = false)
    private UUID storekeeperId;

    @Column(name = "notes", length = 500)
    private String notes;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    public enum IssuanceStatus {
        ACTIVE,           // активная выдача (инвентарь у спортсмена)
        RETURNED,         // инвентарь возвращен
        OVERDUE,          // просрочен возврат
        EXTENDED           // срок возврата продлен
    }
}
```

```
}  
}
```

Листинг 5 – DTO IssuanceRequestDto.java

```
package ru.mirea.asu.dto;  
  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.NotNull;  
import jakarta.validation.constraints.Size;  
import lombok.Data;  
  
import java.time.LocalDate;  
import java.util.UUID;  
  
@Data  
public class IssuanceRequestDto {  
  
    @NotNull(message = "ID спортсмена обязателен")  
    private UUID athleteId;  
  
    @NotBlank(message = "Штрих-код инвентаря обязателен")  
    @Size(min = 5, max = 50, message = "Штрих-код должен содержать от 5 до 50  
символов")  
    private String barcode;  
  
    @NotNull(message = "Дата планируемого возврата обязательна")  
    private LocalDate plannedReturnDate;  
  
    @NotNull(message = "ID кладовщика обязателен")  
    private UUID storekeeperId;  
  
    private String notes;  
}
```

Листинг 6 – DTO IssuanceResponseDto.java

```
package ru.mirea.asu.dto;  
  
import lombok.Builder;  
import lombok.Data;  
  
import java.time.LocalDate;  
import java.util.UUID;  
  
@Data  
@Builder  
public class IssuanceResponseDto {  
  
    private UUID issuanceId;  
    private UUID athleteId;  
    private String athleteName;  
    private UUID inventoryItemId;  
    private String inventoryName;  
    private String barcode;  
    private LocalDate issuanceDate;  
    private LocalDate plannedReturnDate;  
    private String status;  
    private String message;  
}
```

Листинг 7 – Репозиторий `AthleteRepository.java`

```
package ru.mirea.asu.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import ru.mirea.asu.entity.Athlete;

import java.util.Optional;
import java.util.UUID;

public interface AthleteRepository extends JpaRepository<Athlete, UUID> {

    Optional<Athlete> findByEmail(String email);

    @Query("SELECT a.hasActiveDebt FROM Athlete a WHERE a.id = :athleteId")
    boolean hasActiveDebt(@Param("athleteId") UUID athleteId);
}
```

Листинг 8 – Репозиторий `InventoryItemRepository.java`

```
package ru.mirea.asu.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.transaction.annotation.Transactional;
import ru.mirea.asu.entity.InventoryItem;

import java.util.Optional;
import java.util.UUID;

public interface InventoryItemRepository extends JpaRepository<InventoryItem,
UUID> {

    Optional<InventoryItem> findByBarcode(String barcode);

    @Modifying
    @Transactional
    @Query("UPDATE InventoryItem i SET i.status = :status, i.currentHolderId =
:holderId " +
        "WHERE i.barcode = :barcode")
    int updateStatusAndHolder(@Param("barcode") String barcode,
        @Param("status") InventoryItem.InventoryStatus
status,
        @Param("holderId") UUID holderId);

    @Query("SELECT i.status FROM InventoryItem i WHERE i.barcode = :barcode")
    InventoryItem.InventoryStatus getStatusByBarcode(@Param("barcode") String
barcode);
}
```

Листинг 9 – Репозиторий `IssuanceDocumentRepository.java`

```
package ru.mirea.asu.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import ru.mirea.asu.entity.IssuanceDocument;

import java.util.List;
```

```

import java.util.UUID;

public interface IssuanceDocumentRepository extends
JpaRepository<IssuanceDocument, UUID> {

    List<IssuanceDocument> findByAthleteId(UUID athleteId);

    List<IssuanceDocument> findByInventoryItemId(UUID inventoryItemId);

    @Query("SELECT COUNT(d) > 0 FROM IssuanceDocument d " +
        "WHERE d.athlete.id = :athleteId " +
        "AND d.status IN ('ACTIVE', 'OVERDUE')")
    boolean hasActiveIssuances(@Param("athleteId") UUID athleteId);
}

```

Листинг 10 – Исключение AthleteHasDebtException.java

```

package ru.mirea.asu.exception;

import java.util.UUID;

public class AthleteHasDebtException extends RuntimeException {

    public AthleteHasDebtException(String message) {
        super(message);
    }

    public AthleteHasDebtException(UUID athleteId) {
        super("Спортсмен с ID " + athleteId + " имеет активную задолженность. " +
            "Выдача нового инвентаря невозможна до возврата предыдущего.");
    }
}

```

Листинг 11 – Исключение InventoryItemNotFoundException.java

```

package ru.mirea.asu.exception;

import java.util.UUID;

public class InventoryItemNotFoundException extends RuntimeException {

    public InventoryItemNotFoundException(String barcode) {
        super("Единица инвентаря со штрих-кодом '" + barcode + "' не найдена в системе.");
    }

    public InventoryItemNotFoundException(UUID id) {
        super("Единица инвентаря с ID " + id + " не найдена.");
    }
}

```

Листинг 12 – Интерфейс сервиса IssuanceService.java

```

package ru.mirea.asu.service;

import ru.mirea.asu.dto.IssuanceRequestDto;
import ru.mirea.asu.dto.IssuanceResponseDto;

import java.util.UUID;

public interface IssuanceService {

```

```

    /**
     * Выполнить выдачу инвентаря спортсмену
     * @param request DTO с данными для выдачи
     * @return DTO с результатом выдачи
     * @throws ru.mirea.asu.exception.AthleteHasDebtException если спортсмен
    имеет задолженность
     * @throws ru.mirea.asu.exception.InventoryItemNotFoundException если
    инвентарь не найден
     */
    IssuanceResponseDto issueInventory(IssuanceRequestDto request);

    /**
     * Зарегистрировать возврат инвентаря
     * @param issuanceId ID документа выдачи
     * @return обновленный документ выдачи
     */
    IssuanceResponseDto returnInventory(UUID issuanceId);
}

```

Листинг 13 – Реализация сервиса IssuanceServiceImpl.java

```

package ru.mirea.asu.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import ru.mirea.asu.dto.IssuanceRequestDto;
import ru.mirea.asu.dto.IssuanceResponseDto;
import ru.mirea.asu.entity.Athlete;
import ru.mirea.asu.entity.InventoryItem;
import ru.mirea.asu.entity.IssuanceDocument;
import ru.mirea.asu.exception.AthleteHasDebtException;
import ru.mirea.asu.exception.InventoryItemNotFoundException;
import ru.mirea.asu.repository.AthleteRepository;
import ru.mirea.asu.repository.InventoryItemRepository;
import ru.mirea.asu.repository.IssuanceDocumentRepository;

import java.time.LocalDate;
import java.util.UUID;

@Slf4j
@Service
@RequiredArgsConstructor
public class IssuanceServiceImpl implements IssuanceService {

    private final AthleteRepository athleteRepository;
    private final InventoryItemRepository inventoryItemRepository;
    private final IssuanceDocumentRepository issuanceDocumentRepository;

    @Override
    @Transactional
    public IssuanceResponseDto issueInventory(IssuanceRequestDto request) {
        log.info("Начало выдачи инвентаря: спортсмен ID={}, штрих-код={}",
            request.getAthleteId(), request.getBarcode());

        // Шаг 1: Проверка существования спортсмена и наличия задолженности
        Athlete athlete = athleteRepository.findById(request.getAthleteId())
            .orElseThrow(() -> new IllegalArgumentException(
                "Спортсмен с ID " + request.getAthleteId() + " не
найден"));
    }
}

```

```

        // Альтернативный поток: блокировка выдачи должнику
        if (athlete.isHasActiveDebt()) {
            issuanceDocumentRepository.hasActiveIssuances(athlete.getId()) {
                log.warn("Попытка выдачи инвентаря спортсмену-должнику: {}",
                    athlete.getFullName());
                throw new AthleteHasDebtException(athlete.getId());
            }
        }

        // Шаг 2: Поиск инвентаря по штрих-коду
        InventoryItem inventoryItem =
            inventoryItemRepository.findByBarcode(request.getBarcode())
                .orElseThrow(() -> new
                    InventoryItemNotFoundException(request.getBarcode()));

        // Шаг 3: Проверка доступности инвентаря
        if (inventoryItem.getStatus() !=
            InventoryItem.InventoryStatus.AVAILABLE) {
            throw new IllegalStateException(
                "Инвентарь со штрих-кодом " + request.getBarcode() +
                " недоступен для выдачи. Текущий статус: " +
                inventoryItem.getStatus());
        }

        // Шаг 4: Создание документа выдачи
        IssuanceDocument issuanceDocument = new IssuanceDocument();
        issuanceDocument.setAthlete(athlete);
        issuanceDocument.setInventoryItem(inventoryItem);
        issuanceDocument.setIssuanceDate(LocalDate.now());
        issuanceDocument.setPlannedReturnDate(request.getPlannedReturnDate());
        issuanceDocument.setStorekeeperId(request.getStorekeeperId());
        issuanceDocument.setNotes(request.getNotes());
        issuanceDocument.setStatus(IssuanceDocument.IssuanceStatus.ACTIVE);

        IssuanceDocument savedDocument =
            issuanceDocumentRepository.save(issuanceDocument);

        // Шаг 5: Обновление статуса инвентаря и привязка к спортсмену
        inventoryItem.setStatus(InventoryItem.InventoryStatus.ISSUED);
        inventoryItem.setCurrentHolderId(athlete.getId());
        inventoryItemRepository.save(inventoryItem);

        log.info("Выдача инвентаря успешно завершена. ID документа: {}",
            savedDocument.getId());

        // Формирование ответа
        return IssuanceResponseDto.builder()
            .issuanceId(savedDocument.getId())
            .athleteId(athlete.getId())
            .athleteName(athlete.getFullName())
            .inventoryItemId(inventoryItem.getId())
            .inventoryName(inventoryItem.getProductName())
            .barcode(inventoryItem.getBarcode())
            .issuanceDate(savedDocument.getIssuanceDate())
            .plannedReturnDate(savedDocument.getPlannedReturnDate())
            .status(savedDocument.getStatus().name())
            .message("Инвентарь успешно выдан спортсмену " +
                athlete.getFullName())
            .build();
    }

    @Override
    @Transactional
    public IssuanceResponseDto returnInventory(UUID issuanceId) {

```



```

        log.info("Регистрация возврата инвентаря по документу: {}",
issuanceId);

        IssuanceDocument issuanceDocument =
issuanceDocumentRepository.findById(issuanceId)
                .orElseThrow(() -> new IllegalArgumentException(
                        "Документ выдачи с ID " + issuanceId + " не найден"));

        // Проверка, что инвентарь еще не возвращен
        if (issuanceDocument.getStatus() ==
IssuanceDocument.IssuanceStatus.RETURNED) {
            throw new IllegalStateException("Инвентарь уже был возвращен");
        }

        // Обновление документа выдачи
        issuanceDocument.setActualReturnDate(LocalDate.now());
        issuanceDocument.setStatus(IssuanceDocument.IssuanceStatus.RETURNED);
        issuanceDocumentRepository.save(issuanceDocument);

        // Обновление статуса инвентаря
        InventoryItem inventoryItem = issuanceDocument.getInventoryItem();
        inventoryItem.setStatus(InventoryItem.InventoryStatus.AVAILABLE);
        inventoryItem.setCurrentHolderId(null);
        inventoryItemRepository.save(inventoryItem);

        log.info("Возврат инвентаря успешно зарегистрирован. Штрих-код: {}",
                inventoryItem.getBarcode());

        return IssuanceResponseDto.builder()
                .issuanceId(issuanceDocument.getId())
                .athleteId(issuanceDocument.getAthlete().getId())
                .athleteName(issuanceDocument.getAthlete().getFullName())
                .inventoryItemId(inventoryItem.getId())
                .inventoryName(inventoryItem.getProductName())
                .barcode(inventoryItem.getBarcode())
                .issuanceDate(issuanceDocument.getIssuanceDate())
                .plannedReturnDate(issuanceDocument.getActualReturnDate())
                .status(issuanceDocument.getStatus().name())
                .message("Инвентарь успешно возвращен на склад")
                .build();
    }
}

```

Листинг 14 – REST-контроллер *IssuanceController.java*

```

package ru.mirea.asu.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ru.mirea.asu.dto.IssuanceRequestDto;
import ru.mirea.asu.dto.IssuanceResponseDto;
import ru.mirea.asu.exception.AthleteHasDebtException;
import ru.mirea.asu.exception.InventoryItemNotFoundException;
import ru.mirea.asu.service.IssuanceService;

import java.util.UUID;

@Slf4j
@RestController

```

```

@RequestMapping("/api/issuance")
@RequiredArgsConstructor
public class IssuanceController {

    private final IssuanceService issuanceService;

    /**
     * Выдача инвентаря спортсмену
     * POST /api/issuance/issue
     *
     * @param request DTO с данными для выдачи
     * @return DTO с результатом выдачи
     */
    @PostMapping("/issue")
    public ResponseEntity<IssuanceResponseDto> issueInventory(
        @Valid @RequestBody IssuanceRequestDto request) {

        log.info("POST /api/issuance/issue - получен запрос на выдачу");

        IssuanceResponseDto response =
issuanceService.issueInventory(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    /**
     * Регистрация возврата инвентаря
     * POST /api/issuance/return/{issuanceId}
     *
     * @param issuanceId ID документа выдачи
     * @return DTO с результатом возврата
     */
    @PostMapping("/return/{issuanceId}")
    public ResponseEntity<IssuanceResponseDto> returnInventory(
        @PathVariable UUID issuanceId) {

        log.info("POST /api/issuance/return/{issuanceId} - регистрация возврата",
issuanceId);

        IssuanceResponseDto response =
issuanceService.returnInventory(issuanceId);
        return ResponseEntity.ok(response);
    }

    /**
     * Обработчик исключения "Спортсмен-должник"
     * Возвращает HTTP 409 Conflict
     */
    @ExceptionHandler(AthleteHasDebtException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ResponseEntity<ErrorResponse>
handleAthleteHasDebt(AthleteHasDebtException ex) {
        log.error("Ошибка выдачи: {}", ex.getMessage());
        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(new ErrorResponse("DEBTOR_BLOCKED", ex.getMessage()));
    }

    /**
     * Обработчик исключения "Инвентарь не найден"
     * Возвращает HTTP 404 Not Found
     */
    @ExceptionHandler(InventoryItemNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)

```

```

        public ResponseEntity<ErrorResponse>
handleInventoryNotFound(InventoryItemNotFoundException ex) {
    log.error("Ошибка выдачи: {}", ex.getMessage());
    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(new ErrorResponse("INVENTORY_NOT_FOUND",
ex.getMessage()));
}

/**
 * Внутренний класс для форматирования ошибок
 */
record ErrorResponse(String code, String message) {}
}

```

Листинг 15 – Конфигурация BarcodeConfig.java (для работы со штрих-кодами)

```

package ru.mirea.asu.config;

import com.google.zxing.BarcodeFormat;
import com.google.zxing.EncodeHintType;
import com.google.zxing.MultiFormatWriter;
import com.google.zxing.Writer;
import com.google.zxing.client.j2se.MatrixToImageWriter;
import com.google.zxing.common.BitMatrix;
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.awt.image.BufferedImage;
import java.util.HashMap;
import java.util.Map;

@Configuration
public class BarcodeConfig {

    /**
     * Создание генератора штрих-кодов формата Code-128
     * (стандарт для складского учета)
     */
    @Bean
    public Writer barcodeWriter() {
        return new MultiFormatWriter();
    }

    /**
     * Настройки по умолчанию для генерации штрих-кодов
     */
    @Bean
    public Map<EncodeHintType, Object> barcodeHints() {
        Map<EncodeHintType, Object> hints = new HashMap<>();
        hints.put(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.L);
        hints.put(EncodeHintType.MARGIN, 2);
        hints.put(EncodeHintType.CHARACTER_SET, "UTF-8");
        return hints;
    }

    /**
     * Утилитарный метод для генерации изображения штрих-кода
     * Может быть использован для печати этикеток
     */
    public static BufferedImage generateBarcodeImage(String barcodeData, int
width, int height) throws Exception {

```

```

        MultiFormatWriter writer = new MultiFormatWriter();
        Map<EncodeHintType, Object> hints = new HashMap<>();
        hints.put (EncodeHintType.MARGIN, 2);
        hints.put (EncodeHintType.CHARACTER_SET, "UTF-8");

        BitMatrix bitMatrix = writer.encode (barcodeData,
BarcodeFormat.CODE_128, width, height, hints);

        return MatrixToImageWriter.toBufferedImage (bitMatrix);
    }
}

```

Листинг 16 – Конфигурация приложения application.yml

```

spring:
  application:
    name: asu-inventory-management

  datasource:
    url: jdbc:postgresql://localhost:5432/asu_inventory
    username: ${DB_USERNAME:asu_user}
    password: ${DB_PASSWORD:secure_password}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 5
      connection-timeout: 30000

  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
      jdbc:
        batch_size: 20
        format_sql: true
      show-sql: false

  jackson:
    serialization:
      write-dates-as-timestamps: false
    date-format: yyyy-MM-dd

server:
  port: 8080
  servlet:
    context-path: /api

logging:
  level:
    ru.mirea.asu: DEBUG
    org.springframework.web: INFO
    org.hibernate.SQL: WARN

```

Листинг 17 – Файл application.properties для настройки на Astra Linux

```

# Конфигурация для развертывания на Astra Linux Special Edition
server.port=8080
spring.application.name=asu-inventory

# Подключение к PostgreSQL (Postgres Pro Enterprise)
spring.datasource.url=jdbc:postgresql://localhost:5432/asu_inventory

```

```
spring.datasource.username=asu_user
spring.datasource.password=${ASU_DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA настройки
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.jdbc.lob.non_contextual_creation=true

# Логирование на русском
logging.file.path=/var/log/asu-inventory
logging.level.ru.mirea.asu=INFO

# Настройки для работы со сканерами штрих-кодов (через последовательный порт)
barcode.scanner.port=/dev/ttyUSB0
barcode.scanner.baud-rate=9600
```